

[← BACK TO HOME](#)

API Documentation

Welcome to the API documentation and specification of the LRCLIB's API!

This API has a generous rate limiting in place and is openly accessible to all users and applications. There is no need for an API key or any kind of registering!

Implementation Requirements

Please implement your client responsibly to avoid overloading the server. LRCLIB is a free service without API keys, so it relies on clients behaving considerately.

Header

If you are developing an application to interact with LRCLIB, we require you to identify your client in requests. Set the ``User-Agent`` header with your application's name, version, and a link to its homepage or project page or an email address.

For example:

```
LRCGET v0.2.0 (https://github.com/tranxuanthang/lrcget)
```

If modifying ``User-Agent`` is not possible (for example, browser JavaScript restricts this header), you can use ``X-User-Agent`` or ``Lrclib-Client`` as alternatives.

Rate Limiting

When you exceed the rate limit, the server responds with ``429 Too Many Requests`` and a ``Retry-After`` header specifying how many seconds to wait. Your client **must** honor this header. Ignoring it and continuing to send requests may result in a temporary ban.

```
{
  "code": 429,
  "name": "TooManyRequests",
  "message": "Rate limit exceeded"
}
```

Request Throttling

To avoid hitting rate limits, send requests sequentially by waiting for each one to finish before starting the next. Additionally, add a short delay between requests (200–

500 ms), especially for batch operations like scanning a full music library.

Supported Endpoints

Get lyrics with a track's signature

GET /api/get

Attempt to find the best match of lyrics for the track. You must provide the track title and the artist name. Providing the album name and the track's duration in seconds is recommended and improves the matching precision.

If no match is found, the API returns ``404 Not Found``. Missing tracks can be picked up by the background lyrics fetching service and may become available in a later request.

Note: The provided ``duration`` is crucial. LRCLIB will attempt to provide the lyrics only when the duration matches the record in LRCLIB's database, or at least with a difference of ± 2 seconds in duration.

Query parameters

Field	Required	Type	Description
track_name	true	string	Title of the track
artist_name	true	string	Name of the artist
album_name	false	string	Name of the album (recommended)
duration	false	number	Track's duration in seconds (must be between 1 and 3600)

Example request

```
GET /api/get?artist_name=Borislav+Slavov&track_name=I+Want+to+Live&album_name=Baldur's+Gate+3+(Original+Game+Soundtrack)&duration=233
```

Example response

200 OK:

```
{
  "id": 3396226,
  "name": "I Want to Live",
  "trackName": "I Want to Live",
  "artistName": "Borislav Slavov",
  "albumName": "Baldur's Gate 3 (Original Game Soundtrack)",
  "duration": 233,
  "instrumental": false,
```

```
"plainLyrics": "I feel your breath upon my neck\n...The clock won't  
"syncdLyrics": "[00:17.12] I feel your breath upon my neck\n...[00:17.12]  
"lyricsfile": "version: '1.0'\nmetadata:\n  title: I Want to Live\n}"
```

The `lyricsfile` field is a raw Lyricsfile YAML document. Every lyrics record in LRCLIB is guaranteed to include a Lyricsfile, so this field is always present. The `name` field is an alias of `trackName`.

404 Not Found:

```
{  
  "code": 404,  
  "name": "TrackNotFound",  
  "message": "Failed to find specified track"  
}
```

Get lyrics by LRCLIB's ID

GET `/api/get/`

Get a lyrics record by the track's absolute ID. Track IDs are returned by other APIs, such as `/api/get` and `/api/search`.

URL parameters

Field	Required	Type	Description
id	true	number	ID of the track

Example request

```
GET /api/get/3396226
```

Example response

Please see the `/api/get`'s example response.

Search for lyrics records

GET `/api/search`

Search for lyrics records using keywords. This API returns an array of lyrics records that match the specified search condition(s).

At least ONE of the two parameters, `q`` OR `track_name``, must be present. If neither is provided, an empty array is returned.

Note: This API currently returns a maximum of 20 results and does not support pagination. These limitations are subject to change in the future.

Query parameters

Field	Required	Type	Description
q	conditional	string	Search for keyword present in ANY fields (track's title, artist name or album name). When provided, it takes precedence over the structured parameters
track_name	conditional	string	Search for keyword in track's title
artist_name	false	string	Search for keyword in track's artist name
album_name	false	string	Search for keyword in track's album name

Example request

Search for lyrics by using only `q`` parameter:

```
GET /api/search?q=still+alive+portal
```

Search for lyrics by using multiple fields:

```
GET /api/search?track_name=22&artist_name=taylor+swift
```

Response

JSON array of the lyrics records with the following parameters: `id``, `name``, `trackName``, `artistName``, `albumName``, `duration``, `instrumental``, `plainLyrics``, `syncdLyrics`` and `lyricsfile``. Every lyrics record in LRCLIB is guaranteed to include a Lyricsfile, so the `lyricsfile`` field is always present. The `name`` field is an alias of `trackName``.

Publish a new lyrics

POST /api/publish

Note: This API is experimental and subject to potential changes in the future.

Publish a new lyrics to LRCLIB database. This API can be called anonymously, and no registration is required.

If `plainLyrics``, `syncdLyrics`` and `lyricsfile`` are all left empty, the track

will be marked as instrumental.

If a ``lyricsfile`` is provided, it is stored as-is and takes precedence: any ``plainLyrics`` and ``syncdLyrics`` in the same request are ignored.

All previous revisions of the lyrics will still be kept when publishing lyrics for a track that already has existing lyrics.

Obtaining the Publish Token

Every ``POST /api/publish`` request must include a fresh, valid Publish Token in the ``X-Publish-Token`` header. Each Publish Token can only be used once.

The Publish Token consists of two parts: a ``prefix`` and a ``nonce`` concatenated with a colon (``{prefix}:{nonce}``).

To obtain a ``prefix``, you need to make a request to the ``POST /api/request-challenge`` API. This will provide you with a fresh prefix string and a target string.

To find a valid ``nonce``, you must solve a proof-of-work cryptographic challenge using the provided ``prefix`` and ``target``. For implementation examples, please refer to [the source code of LRCGET](#).

Request header

Header name	Required	Description
X-Publish-Token	true	A Publish Token that can be retrieved via solving a cryptographic challenge

Request JSON body parameters

Field	Required	Type	Description
trackName	true	string	Title of the track
artistName	true	string	Track's artist name
albumName	true	string	Track's album name
duration	true	number	Track's duration
plainLyrics	false	string	Plain lyrics for the track
syncdLyrics	false	string	Synchronized lyrics for the track
lyricsfile	false	string	Raw Lyricsfile YAML for the track. When provided, <code>`plainLyrics`</code> and <code>`syncdLyrics`</code> are ignored

Response

Success response: 201 Created

Failed response (incorrect Publish Token):

```
{
  "code": 400,
  "name": "IncorrectPublishTokenError",
  "message": "The provided publish token is incorrect"
}
```

Flag a lyrics

POST /api/flag

Report the currently published lyrics of a track (for example, wrong lyrics, wrong track metadata, or a copyright violation). The flag is recorded against the track's current lyrics.

Every `POST /api/flag` request must include a fresh, valid Publish Token in the `X-Publish-Token` header, obtained the same way as described in the "Publish a new lyrics" section.

Request header

Header name	Required	Description
X-Publish-Token	true	A Publish Token that can be retrieved via solving a cryptographic challenge

Request JSON body parameters

Field	Required	Type	Description
trackId	true	number	ID of the track to flag
content	false	string	Optional reason for the flag

Example request

```
POST /api/flag
X-Publish-Token: VXMwW2qPfW2gkCNSl1i708NJkDghtAyU:0000a1b2c3d4e5f6

{
  "trackId": 3396226,
  "content": "The lyrics don't match the audio"
}
```

Response

Success response: 201 Created

Failed response (incorrect Publish Token):

Request a challenge

[illegible]